

MIDTERM SPECIAL

DSA Rapid Prep

The questions that show up almost every time,
with practice links.

Arrays · Binary Search · Linked List · Hashing · Stack · Queue · Heap · Trees · BST · Recursion · Backtracking · Greedy · Bits

by Aaradhya · BigO One

How to use this sheet

Exam: 2 hours · 10 MCQs + coding questions. Here is the plan for tonight.

1 **Recall every ★ question (~3 hrs)**
Don't code yet. Just recall the approach and the time/space complexity for each. Use the one-line ideas on the next pages.

2 **Actually code 8–10 from scratch**
Reverse LL · LCA · Kadane · Valid Parentheses · Sliding Window Max · Subsets · N-Queens · Validate BST · Rotated Search · Merge K Lists.

3 **Skim the MCQ cheat sheet, then sleep**
The last page. Don't cram new stuff after this. A rested brain codes faster than a tired one.

Legend

LC Easy ↗ = opens the problem on LeetCode

GFG Topic ↗ = GeeksforGeeks (search results, so pick the matching problem)

★ = shows up almost every time. Do these first.

Want it explained?
Watch the quick walkthrough on the BigO One YouTube channel (link in the video description).

01 Arrays — 4 MUST-DO

★ Two Sum

HashMap of value → index. For each x, check if (target − x) is already stored.

`O(n) time · O(n) space`

LC Easy ↗

★ Kadane's (Max Subarray)

Keep a running sum; if it drops below 0, reset it. Track the best sum seen.

`O(n) time · O(1) space`

LC Medium ↗

★ Sort 0s, 1s, 2s (Dutch Flag)

Three pointers low / mid / high. Swap 0s to the front, 2s to the back, single pass.

`O(n) time · O(1) space`

LC Medium ↗

★ Majority Element (Moore's Voting)

Keep candidate + count. Same → +1, different → −1, count 0 → new candidate.

`O(n) time · O(1) space`

LC Easy ↗

THEN, IF TIME: Move zeroes · rotate array (reversal trick) · merge sorted arrays · best time to buy/sell stock · product of array except self · missing/duplicate number · next permutation · subarray sum = K (prefix sum) · trapping rain water · merge intervals

02 Binary Search — 2 MUST-DO

★ Classic BS + First/Last Occurrence

mid = lo + (hi − lo) / 2. Do it iterative AND recursive. For first/last, keep searching after a hit.

`O(log n) time · O(1) space`

LC Classic ↗
LC First / Last ↗

★ Rotated Sorted Array: Search + Find Min

One half is always sorted. Check if target lies in it, else go the other way. For min, compare mid with hi.

`O(log n) time · O(1) space`

LC Search ↗
LC Find Min ↗

THEN, IF TIME: Peak element · sqrt(x) · search insert position · search in 2D matrix · BS on answer: Koko eating bananas, ship capacity, aggressive cows, book allocation

03 Linked List — 4 MUST-DO

★ Reverse Linked List

prev / curr / next pointers, flip curr.next each step. Recursive: reverse the rest, then fix the head.

`Iter O(1) space · Rec O(n) stack`

LC Easy ↗

★ Middle + Cycle Detection + Cycle Start

slow +1, fast +2. If they meet, there is a cycle. Reset one to head, move both +1: they meet at the cycle start.

`O(n) time · O(1) space`

LC Middle ↗
LC Cycle ↗
LC Cycle Start ↗

★ Merge Two Sorted Lists

Dummy node + tail pointer. Attach the smaller head each step, then append the leftover.

`O(n+m) time · O(1) space`

LC Easy ↗

★ Remove Nth Node From End

Dummy node. Move fast n steps ahead, then move both. Slow stops right before the target.

`O(n) time · O(1) space`

LC Medium ↗

THEN, IF TIME: Palindrome LL · intersection of two LLs · delete node given only that node · add two numbers · reverse in k-groups · odd-even list · copy list with random pointer · flatten multilevel LL

04 List / Map / Set (Hashing) — 3 MUST-DO

★ Valid Anagram + Group Anagrams

Count array of 26 letters for valid anagram. For grouping, use the sorted string as the HashMap key.

`O(n) · grouping O(n·k log k)`

LC Valid ↗
LC Group ↗

★ Longest Substring Without Repeating

Sliding window + map of last-seen index. When you see a repeat, jump left past it.

`O(n) time · O(k) space`

LC Medium ↗

★ Top K Frequent Elements

LC Medium ↗

Frequency HashMap, then min-heap of size k (or bucket sort for O(n)).

`O(n log k) time`

THEN, IF TIME: First non-repeating char · contains duplicate · intersection of two arrays · longest consecutive sequence · isomorphic strings · happy number · LRU cache (HashMap + doubly LL)

05 Stack — 3 MUST-DO

★ Valid Parentheses

LC Easy ↗

Push openers. On a closer, the top must be its match. Stack must be empty at the end.

`O(n) time · O(n) space`

★ Next Greater Element (Monotonic Stack)

LC Easy ↗

Keep a decreasing stack. Each new element pops all smaller ones, and those get it as their answer.

`O(n) time · O(n) space`

★ Min Stack (O(1) getMin)

LC Medium ↗

Store (value, min so far) on each push, or keep a second stack of minimums.

`O(1) per operation`

THEN, IF TIME: Infix → postfix · evaluate postfix · daily temperatures · stock span · largest rectangle in histogram · queue using stacks · remove adjacent duplicates · asteroid collision

06 Queue / Deque — 2 MUST-DO

★ Stack using Queues + Circular Queue

LC Stack via Queue ↗
LC Circular Queue ↗

Stack: push, then rotate the queue n−1 times. Circular queue: array + (index + 1) % size, track count.

`Push O(n) · others O(1)`

★ Sliding Window Maximum (Deque)

LC Hard ↗

Deque of indices with decreasing values. Pop back while smaller, pop front when out of the window.

`O(n) time · O(k) space`

THEN, IF TIME: First negative in every window of size k · reverse first k elements of queue · generate binary numbers 1 to n · BFS / level order · rotting oranges · implement deque using array/LL

07 Heap — 3 MUST-DO

★ Kth Largest / Smallest Element

Min-heap of size k. The top is the kth largest. (Quickselect gives avg O(n).)

`O(n log k) time · O(k) space`

LC Medium ↗

★ Merge K Sorted Lists / Arrays

Min-heap holds the current head of each list. Pop the smallest, push its next.

`O(N log k) time`

LC Hard ↗

★ Heap Sort + Build Heap is O(n)

Build max-heap bottom-up (O(n)), then swap root with last and heapify down. Repeat. MCQ favourite!

`O(n log n) time · O(1) space`

LC Practice ↗

GFG Heap Sort Notes ↗

THEN, IF TIME: Median from data stream (two heaps) · top K frequent · K closest points to origin · connect ropes with min cost · sort a k-sorted array · task scheduler · reorganize string

08 Binary Tree — 4 MUST-DO

★ Inorder / Preorder / Postorder

Recursive is easy. Practise ITERATIVE with a stack too (inorder: go left, pop, go right).

`O(n) time · O(h) space`

LC Inorder ↗

LC Preorder ↗

LC Postorder ↗

★ Level Order + Height / Max Depth

BFS with a queue, loop over the current level size. Height = 1 + max(left, right).

`O(n) time`

LC Level Order ↗

LC Max Depth ↗

★ Diameter of Tree + LCA

Diameter: at each node, left height + right height, track the max. LCA: if both sides return non-null, this node is the LCA.

`O(n) time · O(h) space`

LC Diameter ↗

LC LCA ↗

★ Symmetric / Same Tree / Invert Tree

Compare mirror pairs recursively (L.left vs R.right). Invert: swap the children at every node.

`O(n) time`

LC Symmetric ↗
LC Same Tree ↗
LC Invert ↗

THEN, IF TIME: Path sum · right/left view · top/bottom view · zigzag traversal · construct tree from inorder + preorder · check balanced · max path sum · boundary traversal · flatten tree to LL · serialize/deserialize · count nodes

09 BST — 4 MUST-DO

★ Validate BST

Pass a (min, max) range down the recursion. Don't just compare with the children! Or check inorder is strictly increasing.

`O(n) time · O(h) space`

LC Medium ↗

★ Search / Insert / Delete in BST

Go left or right by value. Delete: 0 or 1 child → splice out; 2 children → replace with inorder successor.

`O(h): avg O(log n), worst O(n)`

LC Search ↗
LC Insert ↗
LC Delete ↗

★ Kth Smallest in BST

Inorder of a BST is sorted. Count while traversing and stop at k.

`O(h + k) time`

LC Medium ↗

★ LCA in BST

If both values are smaller go left, both larger go right, otherwise the current node is the LCA.

`O(h) time`

LC Medium ↗

THEN, IF TIME: Inorder successor/predecessor · floor and ceil · sorted array → balanced BST · BST from preorder · two sum in BST · range sum · recover BST

10 Recursion — 3 MUST-DO

★ Fibonacci · Factorial · power(x, n) in O(log n)

Fib: naive is O(2^n), memo or loop gives O(n). Power: square the base, halve n. Always write the base case first.

`Power: O(log n)`

LC Fibonacci ↗
LC Pow(x, n) ↗

★ Subsets (Power Set) + Permutations

Subsets: include / exclude each element → 2^n. Permutations: swap in place or use a used[] array → n!

`O(2^n) · O(n!)`

LC Subsets ↗
LC Permutations ↗
GFG String Perms ↗

★ Tower of Hanoi

Move n−1 to aux, move the largest to target, move n−1 onto it. Total moves = 2^n − 1.

`O(2^n) time`

GFG Tower of Hanoi ↗

THEN, IF TIME: Sum of digits · palindrome check · reverse string/array · GCD · generate parentheses · Josephus problem · dry-run output questions (very common in MCQs!)

11 Backtracking — 3 MUST-DO

★ Subsets I & II · Permutations I & II

Choose → explore → un-choose. With duplicates: sort first and skip a value equal to the previous one at the same level.

`O(2^n) · O(n!)`

LC Subsets ↗
LC Subsets II ↗
LC Perms ↗
LC Perms II ↗

★ N-Queens

Place one queen per row. Track used columns and both diagonals (r − c and r + c) in sets or arrays.

`~O(n!) time`

LC Hard ↗

★ Combination Sum I & II

I: reuse allowed, recurse with the same index. II: each used once, go to i+1 and skip duplicates.

`Exponential`

LC Sum I ↗
LC Sum II ↗

THEN, IF TIME: Word search · palindrome partitioning · letter combos of phone number · rat in a maze · Sudoku solver · m-coloring · subset sum

12 Greedy — 3 MUST-DO

★ Activity Selection / N Meetings in One Room

Sort by END time. Pick the next activity whose start is at or after the last picked end.

`O(n log n) time`

GFG Activity Selection ↗
GFG N Meetings ↗

★ Fractional Knapsack

Sort by value / weight, descending. Take whole items, then a fraction of the last one.

`O(n log n) time`

GFG Fractional Knapsack ↗

★ Jump Game I & II

I: track the farthest reachable index, fail if i > reach. II: greedy windows (BFS style), count jumps.

`O(n) time · O(1) space`

LC Jump I ↗
LC Jump II ↗

THEN, IF TIME: Job sequencing · min coins (and when greedy FAILS!) · min platforms · gas station · assign cookies · non-overlapping intervals · Huffman coding · candy · lemonade change

13 Bit Manipulation

4 MUST-DO

★ Check / Set / Clear / Toggle ith Bit

check: n & (1<<i) set: n | (1<<i) clear: n & ~(1<<i) toggle: n ^ (1<<i)

`O(1)`

GFG Bit Ops ↗

★ Count Set Bits (Brian Kernighan)

n = n & (n − 1) wipes the lowest set bit. The number of loops = number of set bits.

`O(set bits)`

LC Easy ↗

★ Power of Two

n > 0 and (n & (n − 1)) == 0. A power of two has exactly one set bit.

`O(1)`

LC Easy ↗

★ Single Number (XOR Trick) + Missing Number

XOR cancels pairs: a ^ a = 0, a ^ 0 = a. For missing, XOR all indices with all values.

`O(n) time · O(1) space`

LC Single ↗
LC Missing ↗

THEN, IF TIME: Single number II & III · swap without temp · reverse bits · generate subsets using bitmask · Hamming distance · XOR from 1 to n · divide two integers without /

MCQ Cheat Sheet

Memorize these. 10 MCQs = easy marks.

Binary search
`O(log n)`

Heap insert / delete
`O(log n)`

Build heap
`O(n)`

Heap sort
`O(n log n)`

Quick sort worst case
$O(n^2)$

Merge sort
`Always O(n log n)`

BST ops
`avg O(log n) · worst O(n) if skewed`

Inorder of BST
`Sorted order`

Stack / Queue
`LIFO / FIFO`

Recursion
`Uses the call stack. Watch for stack overflow`

HashMap
`avg O(1) · worst O(n)`

Floyd's cycle detection
`O(n) time · O(1) space`

Nodes in a perfect binary tree of height h
`2^(h+1) − 1`

All the best for tomorrow!

Sleep well, read the question twice, dry-run before you submit.

Subscribe to BigO One on YouTube for more DSA.

by Aaradhya